Práctica 3

Teoría de la Computación

Conversión de AFD a Expresiones Regulares

Integrantes:

Marco Antonio Juárez Hipólito
Pérez Flores Arale
Roque Villegas Iván

Conversión de Autómatas a Expresiones Regulares con JFLAP

1.1. Conversión de Autómatas Finitos Deterministas a Expresiones Regulares

Utilizando la herramienta JFLAP, el alumno deberá realizar las siguientes actividades:

1. **Selección y Análisis de Autómatas**

   Los cinco Autómatas Finitos Deterministas (AFD) seleccionados de la Lista 2 son los siguientes:

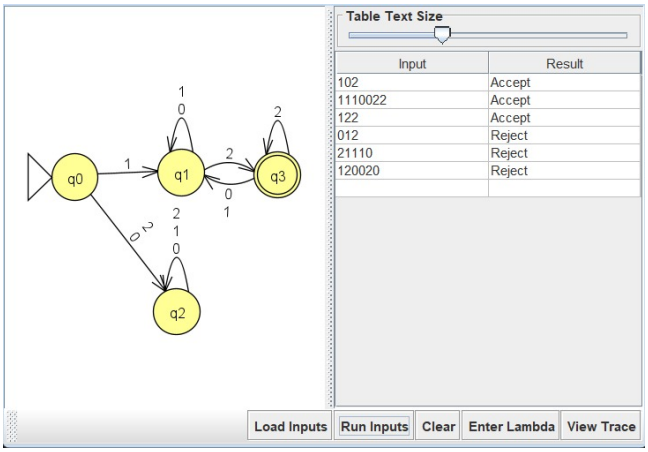
   (a) **Selección y Análisis de Autómatas**

   Los cinco Autómatas Finitos Deterministas (AFD) seleccionados de la Lista 2 son los siguientes:

   - **Ejercicio 94**

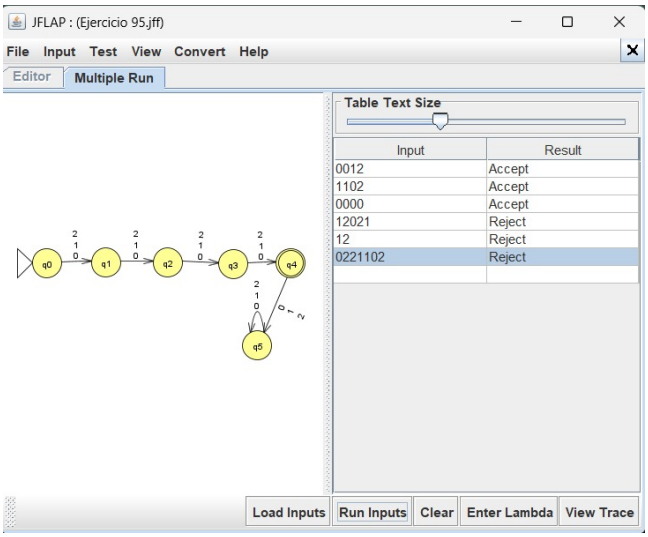
     94. Las palabras de $\Sigma = \{0, 1, 2\}$ que comienzan con 1 y terminan con 2.

Estado	Entrada 0	Entrada 1	Entrada 2
$\rightarrow q_0$	q_2	q_1	q_2
q_1	q_1	q_1	q_3
q_2	q_2	q_2	q_2
$*q_3$	q_1	q_1	q_3



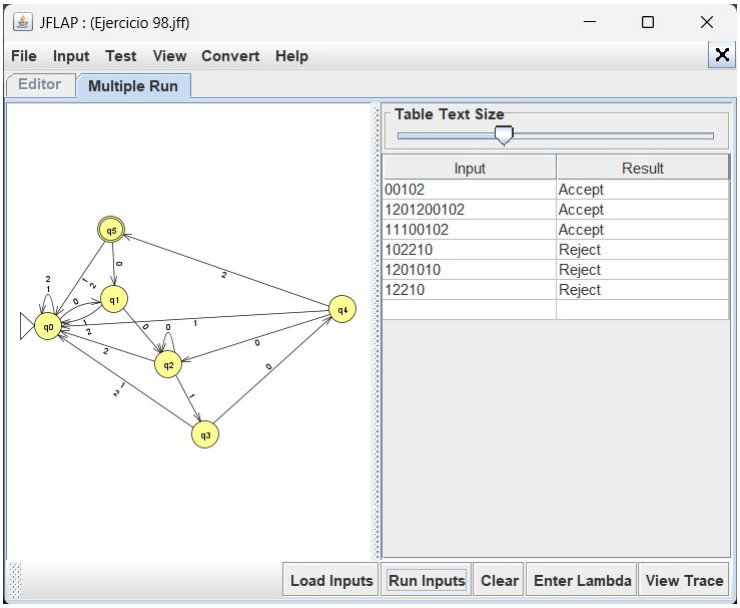
- **Ejercicio 95**
  95. Las palabras de $\Sigma = \{0, 1, 2\}$ que tienen longitud 4.

Estado	**0**	**1**	**2**
$\rightarrow q_0$	q_1	q_1	q_1
q_1	q_2	q_2	q_2
q_2	q_3	q_3	q_3
q_3	q_4	q_4	q_4
$*q_4$	q_5	q_5	q_5
q_5	q_5	q_5	q_5



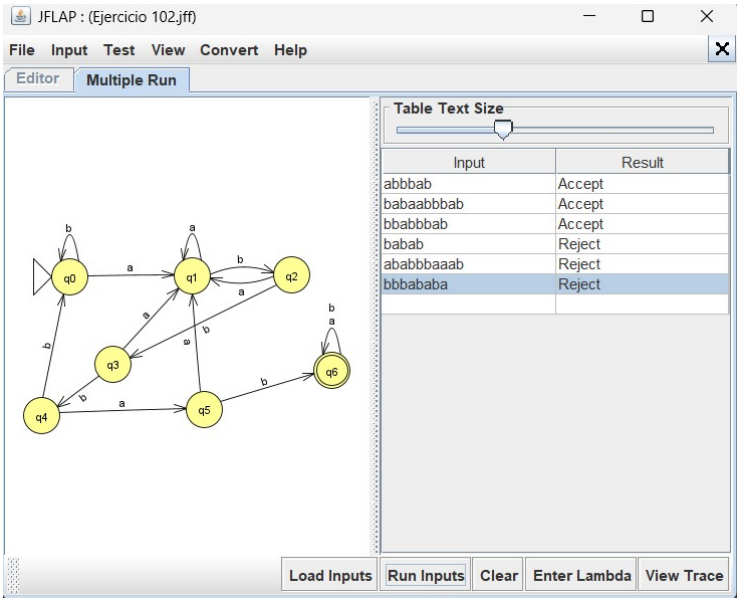
- **Ejercicio 98**
  98. Las palabras de $\Sigma = \{0, 1, 2\}$ que terminan con la cadena 00102.

Estado	**0**	**1**	**2**
$\rightarrow q_0$	q_1	q_0	q_0
q_1	q_2	q_0	q_0
q_2	q_2	q_3	q_0
q_3	q_4	q_0	q_0
q_4	q_2	q_0	q_5
$*q_5$	q_1	q_0	q_0



- **Ejercicio 102**
  102.* Las palabras de $\Sigma = \{a, b\}$ que contienen la cadena `abbbab`.

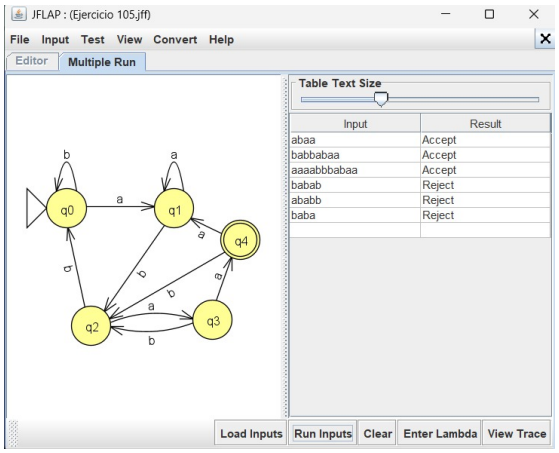
Estado	**a**	**b**
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_4
q_4	q_5	q_0
q_5	q_1	q_6
$*q_6$	q_6	q_6



- **Ejercicio 105**

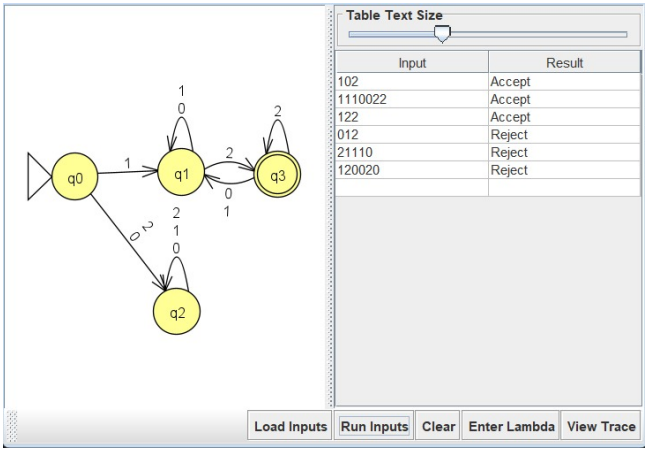
  105. Las palabras de $\Sigma = \{a, b\}$ que terminan con la cadena `abaa`.

Estado	**a**	**b**
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_3	q_0
q_3	q_4	q_2
$*q_4$	q_1	q_2



- **Proceso de Conversión**
  - Para cada autómata seleccionado:
    - Realizar la conversión a expresión regular utilizando el método de eliminación de estados en JFLAP.
    - Documentar paso a paso el proceso de conversión.
    - Capturar la expresión regular resultante.
- **Ejercicio 94**
  94. Las palabras de $\Sigma = \{0, 1, 2\}$ que comienzan con 1 y terminan con 2.

Estado	**Entrada 0**	**Entrada 1**	**Entrada 2**
$\rightarrow q_0$	q_2	q_1	q_2
q_1	q_1	q_1	q_3
q_2	q_2	q_2	q_2
$*q_3$	q_1	q_1	q_3



2. Proceso de Conversión en JFLAP

Para transformar el Autómata Finito Determinista (AFD) en una Expresión Regular (ER), se utilizó la herramienta *Convert FA to RE* de JFLAP, basada en el método de eliminación de estados. El procedimiento seguido fue el siguiente:

i. **Normalización del autómata:** Se añadieron estados auxiliares para garantizar un único estado inicial y un único estado final, conectados mediante transiciones vacías (λ).

ii. **Identificación de estados irrelevantes:** El estado q_2 fue excluido del proceso de conversión, ya que corresponde a un estado de error que no contribuye a la aceptación de cadenas.

iii. **Eliminación de estados intermedios:** Se eliminaron los estados q_1 y q_3 de manera sucesiva. En cada paso, JFLAP

actualizó las transiciones aplicando la regla de eliminación de estados:

$$R_{ij} = R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}$$

donde cada término representa las posibles rutas entre estados antes y después de eliminar el estado intermedio.

3. Expresión Regular Resultante

La expresión regular obtenida mediante JFLAP es:

$$r = 1(0+1)^*2(2+(0+1)(0+1)^*2)^*$$

4. Interpretación de la Expresión

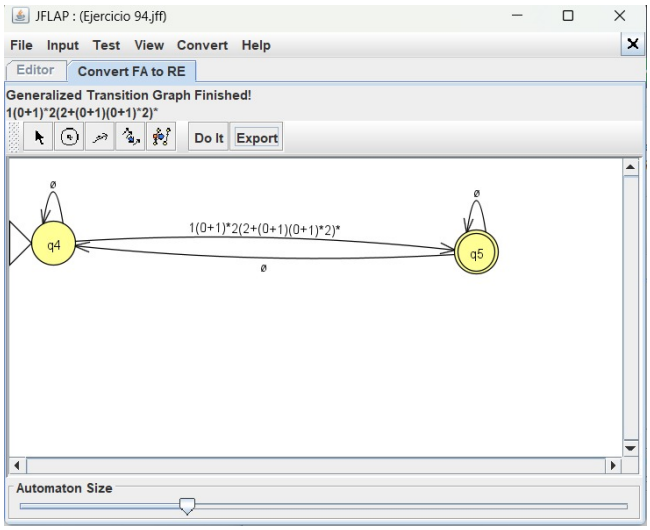
- $1(0+1)^*$: Indica que la cadena debe iniciar con el símbolo 1, seguido de cualquier combinación de 0 y 1.
- 2: Representa la transición que permite alcanzar por primera vez un estado de aceptación.
- $(2+(0+1)(0+1)^*2)^*$: Permite repetir patrones que aseguran que la cadena continúe terminando en 2, ya sea agregando más símbolos 2 o secuencias que vuelven a finalizar en 2.

5. Simplificación

La expresión anterior puede simplificarse a una forma más compacta:

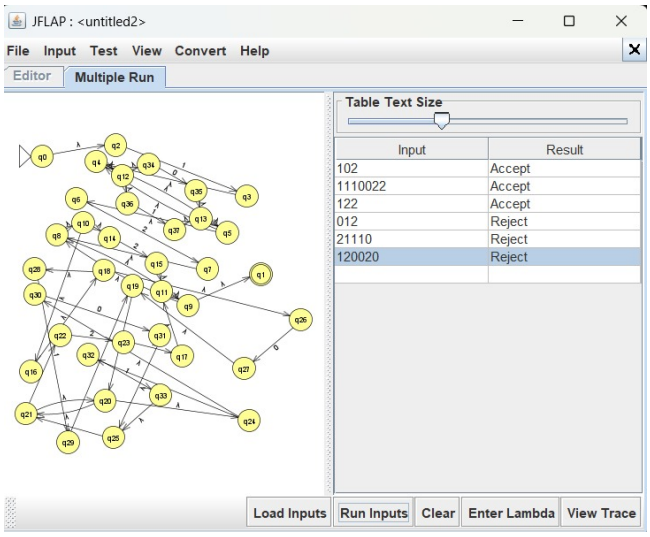
$$r = 1(0+1+2)^*2$$

ya que la parte final de la expresión no modifica la condición fundamental del lenguaje: iniciar con 1 y terminar con 2.



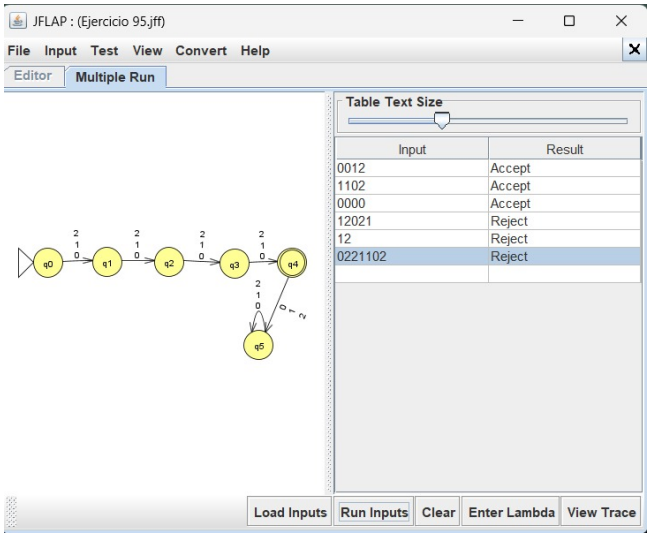
Comparación de equivalencia

Para verificar la equivalencia entre el autómata y la expresión regular obtenida, se realizaron pruebas con distintas cadenas, comprobando que ambos formalismos aceptan y rechazan los mismos casos.



- **Ejercicio 95**
  95. Las palabras de $\Sigma = \{0, 1, 2\}$ que tienen longitud 4.

Estado	Entrada 0	Entrada 1	Entrada 2
$\rightarrow q_0$	q_1	q_1	q_1
q_1	q_2	q_2	q_2
q_2	q_3	q_3	q_3
q_3	q_4	q_4	q_4
$*q_4$	q_5	q_5	q_5
q_5	q_5	q_5	q_5



2. Proceso de Conversión en JFLAP

Para transformar el Autómata Finito Determinista (AFD) en una Expresión Regular (ER), se utilizó la herramienta *Convert FA to RE* de JFLAP, basada en el método de eliminación de estados. El procedimiento seguido fue el siguiente:

i. **Normalización del autómata:** Se añadieron estados auxiliares para garantizar un único estado inicial y un único estado final, conectados mediante transiciones vacías (λ).

ii. **Eliminación de estados intermedios:** Se eliminaron de forma sucesiva los estados q_1, q_2, q_3 y q_5. En cada paso, JFLAP actualizó las transiciones aplicando la regla:

$$R_{ij} = R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}$$

iii. **Obtención del resultado:** El proceso continuó hasta quedar únicamente el estado inicial y el estado final, obteniendo así la expresión regular equivalente.

3. Expresión Regular Resultante

La expresión regular obtenida mediante JFLAP es:

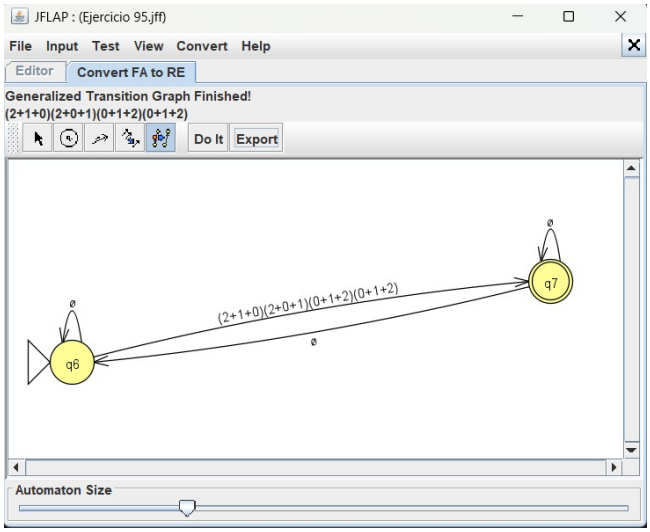
$$r = (0 + 1 + 2)(0 + 1 + 2)(0 + 1 + 2)(0 + 1 + 2)$$

4. Interpretación de la Expresión

- $(0 + 1 + 2)$: Representa cualquier símbolo del alfabeto.
- La repetición de este término cuatro veces indica que la cadena debe tener exactamente cuatro símbolos.

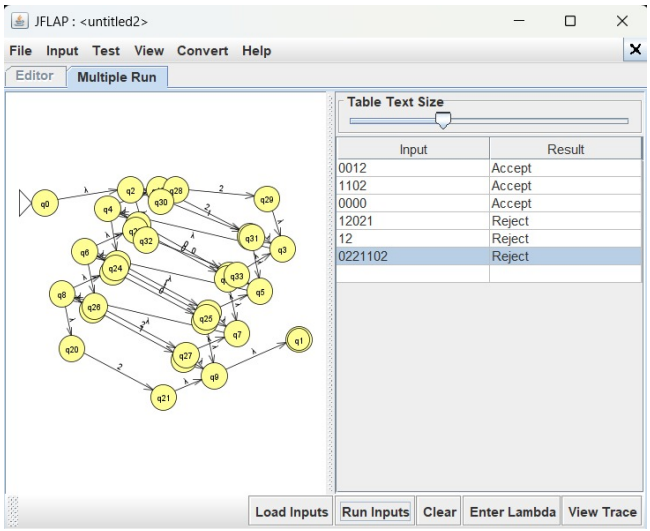
5. Simplificación

La expresión ya se encuentra en su forma más simple, ya que describe directamente cadenas de longitud exacta 4.



Comparación de equivalencia

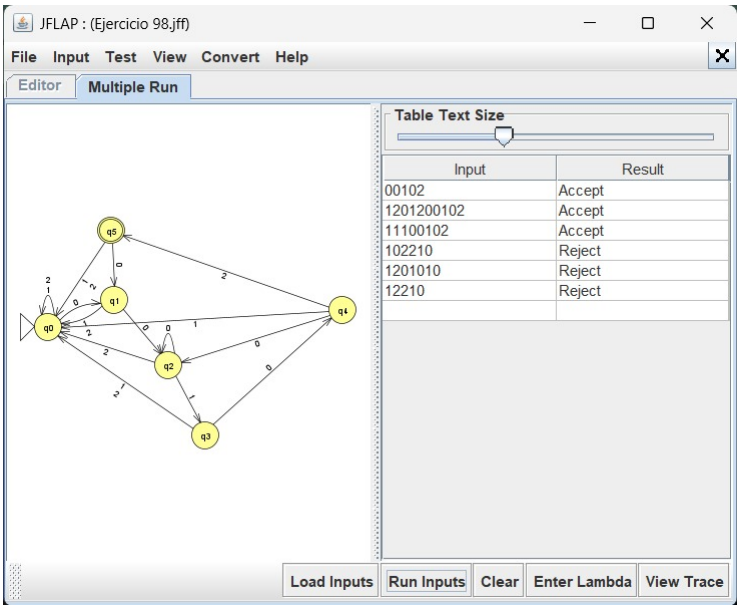
Para verificar la equivalencia entre el autómata y la expresión regular, se probaron diversas cadenas, comprobando que ambas representaciones aceptan únicamente aquellas de longitud 4.



- **Ejercicio 98**

  98. Las palabras de $\Sigma = \{0, 1, 2\}$ que terminan con la cadena `00102`.

Estado	**Entrada 0**	**Entrada 1**	**Entrada 2**
$\rightarrow q_0$	q_1	q_0	q_0
q_1	q_2	q_0	q_0
q_2	q_2	q_3	q_0
q_3	q_4	q_0	q_0
q_4	q_2	q_0	q_5
$*q_5$	q_1	q_0	q_0



2. Proceso de Conversión en JFLAP

Para transformar el Autómata Finito Determinista (AFD) en una Expresión Regular (ER), se utilizó la herramienta *Convert FA to RE* de JFLAP, basada en el método de eliminación de estados. El procedimiento seguido fue el siguiente:

i. **Normalización del autómata:** Se añadieron estados auxiliares para garantizar un único estado inicial y un único estado final.

ii. **Eliminación de estados intermedios:** Se eliminaron de forma sucesiva los estados q_1, q_2, q_3 y q_4, actualizando las transiciones mediante la regla:

$$R_{ij} = R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}$$

iii. **Obtención del resultado:** El proceso continuó hasta obtener una única transición entre el estado inicial y el estado final.

3. Expresión Regular Resultante

La expresión regular obtenida mediante JFLAP es:

$$
\begin{aligned}
r = {} & (1+2)^*0((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^*1 \\
& ((2+1)(1+2)^*0((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^*1)^* \\
& 0((0+1(1+2)^*0((2+1)(1+2)^*0)^*0)(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^* \\
& 1((2+1)(1+2)^*0((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^*1)^*0)^* \\
& 2((0+(2+1)(1+2)^*0)((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^* \\
& 1((2+1)(1+2)^*0((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^*1)^*0 \\
& ((0+1(1+2)^*0((2+1)(1+2)^*0)^*0)(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^* \\
& 1((2+1)(1+2)^*0((2+1)(1+2)^*0)^*0(0+2(1+2)^*0((2+1)(1+2)^*0)^*0)^*1)^*0)^*2)^*
\end{aligned}
$$

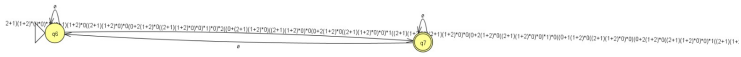
Simplificación

La expresión anterior puede simplificarse a:

$$r = (0+1+2)^*00102$$

4. Interpretación de la Expresión

- $(0+1+2)^*$: Representa cualquier cadena (posiblemente vacía) formada por símbolos del alfabeto.
- 00102: Es la secuencia fija con la que debe terminar toda cadena aceptada.

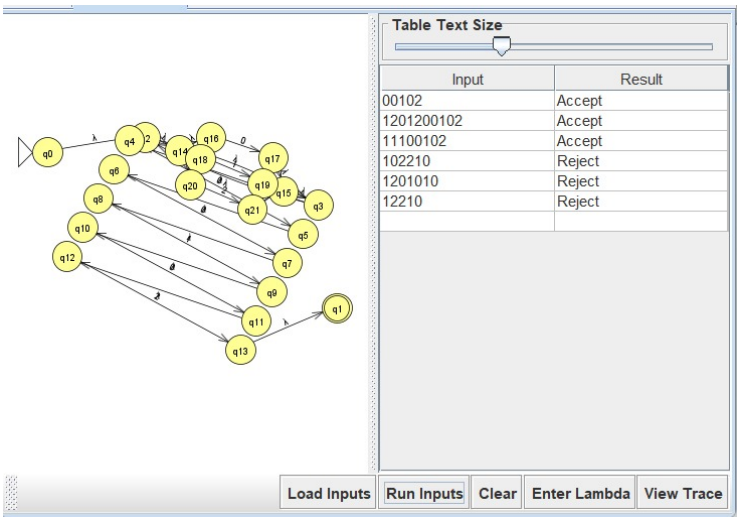


5. Simplificación

La expresión ya se encuentra en su forma más simple, ya que describe directamente todas las cadenas que terminan con la subcadena `00102`.

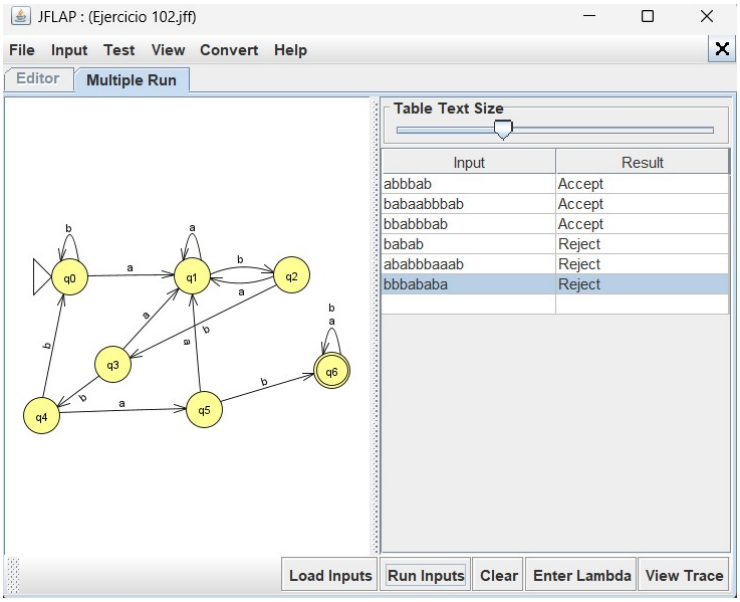
Comparación de equivalencia

Para verificar la equivalencia entre el autómata y la expresión regular, se probaron distintas cadenas, confirmando que ambas representaciones aceptan únicamente aquellas que terminan con la secuencia `00102`.



- **Ejercicio 102**
  102.* Las palabras de $\Sigma = \{a, b\}$ que contienen la cadena `abbbab`.

Estado	**Entrada a**	**Entrada b**
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_4
q_4	q_5	q_0
q_5	q_1	q_6
$*q_6$	q_6	q_6



2. Proceso de Conversión en JFLAP

Para transformar el Autómata Finito Determinista (AFD) en una Expresión Regular (ER), se utilizó la herramienta *Convert FA to RE* de JFLAP, basada en el método de eliminación de estados. El procedimiento seguido fue el siguiente:

i. **Normalización del autómata:** Se añadieron estados auxiliares para garantizar un único estado inicial y un único estado final.

ii. **Eliminación de estados intermedios:** Se eliminaron de forma sucesiva los estados q_1, q_2, q_3, q_4 y q_5, actualizando las transiciones mediante la regla:

$$R_{ij} = R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}$$

iii. **Obtención del resultado:** El proceso continuó hasta obtener una única transición entre el estado inicial y el estado final.

3. Expresión Regular Resultante

La expresión regular obtenida mediante JFLAP es:

$$
\begin{aligned}
r = {} & b^*aa^*b(aa^*b)^*b(aa^*b(aa^*b)^*b)^*b \\
& (bb^*aa^*b(aa^*b)^*b(aa^*b(aa^*b)^*b)^*b)^*a \\
& (aa^*b(aa^*b)^*b(aa^*b(aa^*b)^*b)^*b \\
& (bb^*aa^*b(aa^*b)^*b(aa^*b(aa^*b)^*b)^*b)^*a)^* \\
& b(a+b)^*
\end{aligned}
$$

4. Interpretación de la Expresión

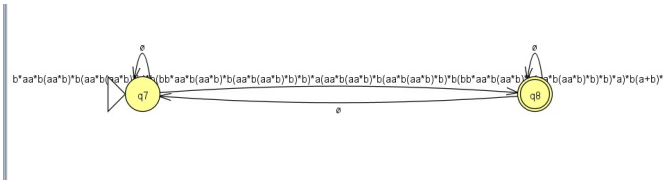
- La expresión generada describe todas las posibles combinaciones de símbolos que contienen la subcadena `abbbab`.
- Debido al proceso de eliminación de estados, la expresión resultante puede ser extensa y no necesariamente simplificada.

5. Simplificación

La expresión anterior es equivalente a la siguiente forma simplificada:

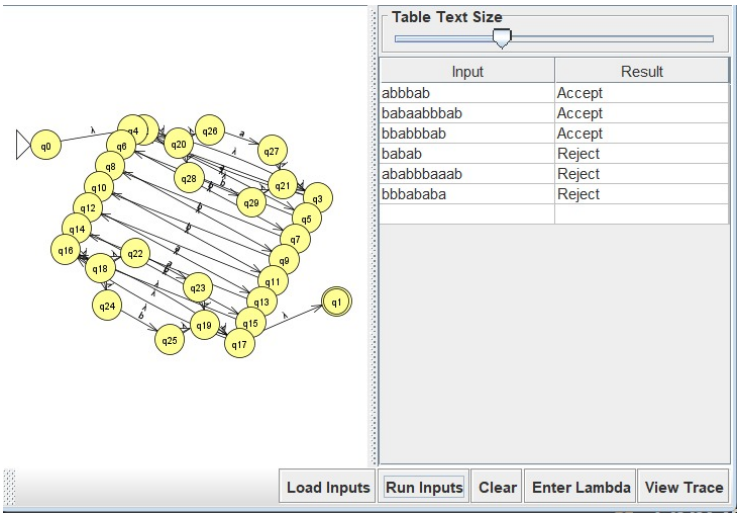
$$r = (a+b)^*abbbab(a+b)^*$$

la cual representa todas las cadenas sobre el alfabeto que contienen la subcadena `abbbab`.



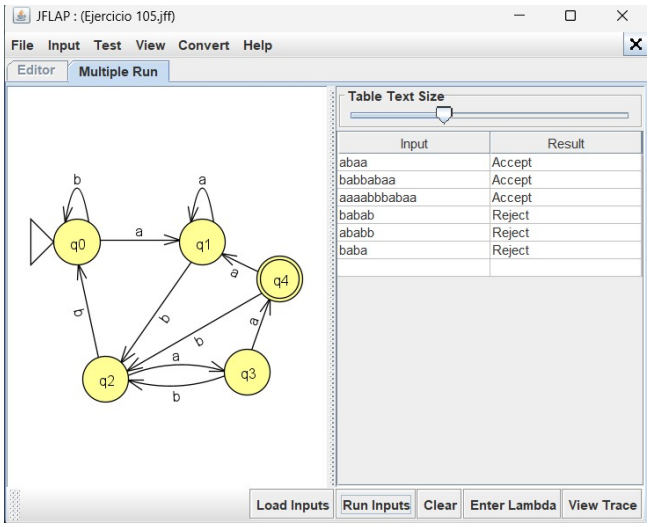
Comparación de equivalencia

Para verificar la equivalencia entre el autómata y la expresión regular, se probaron diversas cadenas, confirmando que ambas representaciones aceptan únicamente aquellas que contienen la subcadena `abbbab`.



- **Ejercicio 105**
  105. Las palabras de $\Sigma = \{a, b\}$ que terminan con la cadena `abaa`.

Estado	**Entrada a**	**Entrada b**
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_3	q_0
q_3	q_4	q_2
$*q_4$	q_1	q_2



2. Proceso de Conversión en JFLAP

Para transformar el Autómata Finito Determinista (AFD) en una Expresión Regular (ER), se utilizó la herramienta *Convert FA to RE* de JFLAP, basada en el método de eliminación de estados. El procedimiento seguido fue el siguiente:

i. **Normalización del autómata:** Se añadieron estados auxiliares para garantizar un único estado inicial y un único estado final.

ii. **Eliminación de estados intermedios:** Se eliminaron de forma sucesiva los estados q_1, q_2 y q_3, actualizando las transiciones mediante la regla:

$$R_{ij} = R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}$$

iii. **Obtención del resultado:** El proceso continuó hasta obtener una única transición entre el estado inicial y el estado final.

3. Expresión Regular Resultante

La expresión regular obtenida mediante JFLAP es:

$$\begin{aligned} r = {} & b^*aa^*b(bb^*aa^*b)^*a(b(bb^*aa^*b)^*a)^*a \\ & ((b + aa^*b)(bb^*aa^*b)^*a(b(bb^*aa^*b)^*a)^*a)^* \end{aligned}$$

4. Interpretación de la Expresión

– La expresión generada describe todas las combinaciones de símbolos que permiten llegar a una cadena que termina con

la secuencia `abaa`.

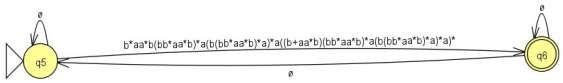
– Debido al proceso de eliminación de estados, la expresión puede ser extensa y poco intuitiva.

5. Simplificación

La expresión anterior es equivalente a la forma simplificada:

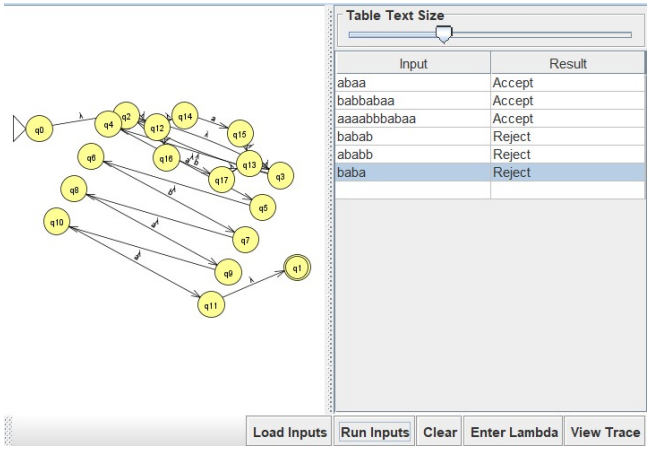
$$r = (a + b)^* abaa$$

la cual representa todas las cadenas sobre el alfabeto que terminan con la subcadena `abaa`.



Comparación de equivalencia

Para verificar la equivalencia entre el autómata y la expresión regular, se probaron diversas cadenas, confirmando que ambas representaciones aceptan únicamente aquellas que terminan con la secuencia `abaa`.



1.2. Comparación de Métodos de Conversión

i. **Método de Eliminación de Estados**
Este método consiste en transformar un autómata finito determinista (AFD) en una expresión regular mediante la eliminación sistemática de sus estados, preservando el lenguaje reconocido en cada paso.
Procedimiento:

A. Si el autómata tiene más de un estado final, se crea un único estado final conectado mediante transiciones λ.
B. Si existen múltiples estados iniciales, se agrega un nuevo estado inicial con transiciones λ hacia los originales.
C. Se reemplazan las etiquetas de las transiciones por expresiones regulares (inicialmente símbolos del alfabeto).
D. Se eliminan los estados intermedios uno por uno. Al eliminar un estado k, se actualizan las transiciones entre los estados restantes usando la fórmula:

$$R_{ij} = R_{ij} + R_{ik}(R_{kk})^* R_{kj}$$

donde:
- R_{ij} es la expresión regular de i a j
- k es el estado que se elimina

E. El proceso continúa hasta que solo queden el estado inicial y el estado final.
F. La expresión regular resultante será la etiqueta de la transición entre el estado inicial y el estado final.

Ventajas:
- Es un método sistemático y fácil de aplicar paso a paso.
- Es adecuado para implementaciones gráficas como en JFLAP.

Desventajas:
- Puede generar expresiones muy largas y difíciles de simplificar.
- El orden de eliminación de estados puede afectar la complejidad del resultado.

ii. **Método de Ecuaciones**
Este método modela el comportamiento del autómata como un sistema de ecuaciones algebraicas, donde cada ecuación representa las cadenas que llevan a un estado.
Procedimiento:

A. Se asigna una variable R_i a cada estado q_i del autómata.
B. Se construyen ecuaciones para cada estado en función de sus transiciones:
   - Si hay una transición de q_i a q_j con símbolo a, se agrega el término aR_j.
C. Si un estado es de aceptación, se añade el término ϵ a su ecuación.
D. Se obtiene un sistema de ecuaciones con expresiones regulares.
E. Se resuelve el sistema despejando la variable correspondiente al estado inicial.

Ejemplo de ecuaciones:

$$R_1 = aR_1 + bR_2$$
$$R_2 = aR_1 + \lambda$$

Ventajas:
- Permite un enfoque matemático estructurado.
- Es útil para analizar formalmente el lenguaje del autómata.

Desventajas:
- Puede ser complicado resolver sistemas grandes.
- Requiere manipulación algebraica de expresiones regulares.

iii. **Método de Arden**
Este método se basa en el Teorema de Arden, el cual permite resolver ecuaciones con variables recursivas en expresiones regulares.
Teorema de Arden:
Si se tiene una ecuación de la forma:

$$R = Q + RP$$

entonces su solución es:

$$R = QP^*$$

siempre que P no contenga λ.
Procedimiento:

A. Se parte de las ecuaciones obtenidas mediante el método de ecuaciones.
B. Se identifica una ecuación donde la variable aparece en ambos lados.
C. Se reescribe en la forma $R = Q + RP$.
D. Se aplica el Teorema de Arden para despejar la variable:

$$R = QP^*$$

E. Se sustituye el resultado en las demás ecuaciones.
F. Se repite el proceso hasta obtener una expresión final.

Ejemplo:

$$R = a + Rb$$

Aplicando Arden:

$$R = ab^*$$

Ventajas:

- Simplifica la resolución de ecuaciones recursivas.
- Reduce significativamente el trabajo algebraico.

Desventajas:

- Solo aplica a ecuaciones en forma específica.
- Requiere experiencia en manipulación de expresiones regulares.

Aplicación y Comparación

i. **Autómata 1 (Ejercicio 94)**
Lenguaje: Palabras sobre $\Sigma = \{0, 1, 2\}$ que comienzan con 1 y terminan con 2.

Estado	**Entrada 0**	**Entrada 1**	**Entrada 2**
$\rightarrow q_0$	q_2	q_1	q_2
q_1	q_1	q_1	q_3
q_2	q_2	q_2	q_2
$*q_3$	q_1	q_1	q_3

Lenguaje reconocido:
El autómata acepta todas las cadenas que comienzan con el símbolo 1 y terminan con el símbolo 2, permitiendo cualquier símbolo del alfabeto en la parte intermedia.

$$R = 1(0 + 1 + 2)^*2$$

ii. **Aplicación del Método de Eliminación de Estados**
Se identifican los estados:

$$q_0 \text{ (inicial)}, \quad q_1, \quad q_2 \text{ (trampa)}, \quad q_3 \text{ (final)}$$

El estado q_2 es un estado trampa, por lo que no contribuye a la aceptación del lenguaje.
Paso 1: Desde el estado inicial q_0, la única transición válida es con el símbolo 1:

$$1 \to q_1$$

Paso 2: En el estado q_1 se pueden leer los símbolos 0 y 1 permaneciendo en el mismo estado, y con 2 se transita al estado final q_3:

$$(0+1)^*2$$

Paso 3: En el estado q_3 se pueden seguir leyendo símbolos del alfabeto, permitiendo regresar a q_1 con 0 o 1, o permanecer en q_3 con 2, lo que genera:

$$(0+1+2)^*$$

Resultado final:

$$R = 1(0+1+2)^*2$$

iii. **Aplicación del Método de Ecuaciones y Arden**
Se asignan variables a cada estado:

$$R_0, R_1, R_2, R_3$$

donde cada R_i representa las cadenas que llevan desde el estado q_i a un estado de aceptación.
Ecuaciones:

$$R_0 = 0R_2 + 1R_1 + 2R_2$$

$$R_1 = 0R_1 + 1R_1 + 2R_3$$

$$R_2 = 0R_2 + 1R_2 + 2R_2$$

$$R_3 = 0R_1 + 1R_1 + 2R_3 + \lambda$$

Resolución:
El estado R_2 es un estado trampa, por lo que no contribuye a la aceptación del lenguaje y se omite en la resolución.
Partimos de la ecuación de R_3:

$$R_3 = (0 + 1)R_1 + 2R_3 + \lambda$$

Reordenamos:

$$R_3 = ((0 + 1)R_1 + \lambda) + 2R_3$$

Aplicando el Teorema de Arden:

$$R_3 = ((0 + 1)R_1 + \lambda)2^*$$

Ahora sustituimos R_3 en la ecuación de R_1:

$$R_1 = (0 + 1)R_1 + 2R_3$$

$$R_1 = (0 + 1)R_1 + 2((0 + 1)R_1 + \lambda)2^*$$

Distribuimos:

$$R_1 = (0 + 1)R_1 + 2(0 + 1)R_1 2^* + 2\lambda 2^*$$

Como $2\lambda = 2$, se tiene:

$$R_1 = (0 + 1)R_1 + 2(0 + 1)R_1 2^* + 22^*$$

Factorizamos R_1:

$$R_1 = [(0+1) + 2(0+1)2^*]\, R_1 + 22^*$$

Desarrollamos la expresión:

$$(0+1) + 2(0+1)2^*$$

$$= 0 + 1 + 2(0+1)2^*$$

$$= 0 + 1 + (20 + 21)2^*$$

$$= 0 + 1 + 20\, 2^* + 21\, 2^*$$

Reagrupamos:

$$= 0 + 1 + 2(02^* + 12^*)$$

$$= 0 + 1 + 2(0+1)2^*$$

Expresamos en términos del alfabeto:

$$= (0+1+2)(\lambda + 02^* + 12^*)$$

Por lo tanto, la ecuación queda:

$$R_1 = (0+1+2)(\lambda + 02^* + 12^*)R_1 + 22^*$$

Aplicamos nuevamente el Teorema de Arden:

$$R_1 = [(0+1+2)(\lambda + 02^* + 12^*)]^*\, 22^*$$

Simplificamos:
Dado que la expresión genera cadenas sobre $\{0, 1, 2\}$, se tiene:

$$[(0+1+2)(\lambda + 02^* + 12^*)]^* = (0+1+2)^*$$

Entonces:

$$R_1 = (0+1+2)^* 22^*$$

$$22^* = 2^+$$

Por lo tanto:

$$R_1 = (0+1+2)^* 2$$

Resultado final:

$$R_0 = 1R_1$$

$$R_0 = 1(0 + 1 + 2)^*2$$

iv. **Comparación de Métodos**

- **Complejidad de la expresión regular:** Ambos métodos producen la misma expresión:

$$1(0+1+2)^*2$$

La expresión es simple y representa correctamente el lenguaje.

- **Tiempo requerido:** El método de eliminación de estados es más rápido, ya que se basa en el análisis directo del autómata. El método de ecuaciones requiere más pasos algebraicos y sustituciones.
- **Facilidad de aplicación:** El método de eliminación de estados es más intuitivo y visual. El método de ecuaciones y Arden es más formal, pero más complejo de aplicar manualmente.

v. **Autómata 2 (Ejercicio 105)**

Lenguaje: Palabras sobre $\Sigma = \{a, b\}$ que terminan con la cadena `abaa`.

Estado	**Entrada a**	**Entrada b**
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_3	q_0
q_3	q_4	q_2
$*q_4$	q_1	q_2

Lenguaje reconocido:

El autómata acepta todas las cadenas que terminan con la subcadena `abaa`, por lo tanto:

$$R = (a+b)^*abaa$$

vi. **Aplicación del Método de Eliminación de Estados**
Se observa que el autómata reconoce cadenas que pueden tener cualquier combinación de símbolos a y b, pero deben finalizar con la secuencia `abaa`.
Paso 1: Desde el estado inicial q_0, se pueden generar cadenas arbitrarias:

$$(a+b)^*$$

Paso 2: La secuencia necesaria para llegar al estado final es:

$$a \to b \to a \to a$$

$$abaa$$

Resultado final:

$$R = (a+b)^* abaa$$

vii. **Aplicación del Método de Ecuaciones y Arden**
Se asignan variables:

$$R_0, R_1, R_2, R_3, R_4$$

Ecuaciones:

$$R_0 = aR_1 + bR_0$$

$$R_1 = aR_1 + bR_2$$

$$R_2 = aR_3 + bR_0$$

$$R_3 = aR_4 + bR_2$$

$$R_4 = aR_1 + bR_2 + \lambda$$

Resolución:
Paso 1: Resolver R_0

$$R_0 = aR_1 + bR_0$$

Aplicando Arden:

$$R_0 = b^* aR_1$$

Paso 2: Resolver R_1

$$R_1 = aR_1 + bR_2$$

Aplicando Arden:

$$R_1 = a^* bR_2$$

Paso 3: Sustituir en R_0

$$R_0 = b^* a(a^* bR_2)$$

$$R_0 = b^* aa^* bR_2$$

Paso 4: Resolver R_2

$$R_2 = aR_3 + bR_0$$

Sustituyendo R_0:

$$R_2 = aR_3 + b(b^* aa^* bR_2)$$

$$R_2 = aR_3 + bb^*aa^*bR_2$$

Factorizamos:

$$R_2 = (bb^*aa^*b)R_2 + aR_3$$

Aplicando Arden:

$$R_2 = (bb^*aa^*b)^*aR_3$$

Paso 5: Resolver R_3

$$R_3 = aR_4 + bR_2$$

Sustituyendo R_2:

$$R_3 = aR_4 + b((bb^*aa^*b)^*aR_3)$$

$$R_3 = aR_4 + b(bb^*aa^*b)^*aR_3$$

Factorizamos:

$$R_3 = [b(bb^*aa^*b)^*a]\, R_3 + aR_4$$

Aplicando Arden:

$$R_3 = [b(bb^*aa^*b)^*a]^*\, aR_4$$

Paso 6: Sustituir en R_2

$$R_2 = (bb^*aa^*b)^*a\, [b(bb^*aa^*b)^*a]^*\, aR_4$$

Paso 7: Sustituir en R_1

$$R_1 = a^*bR_2$$

Paso 8: Sustituir en R_0

$$R_0 = b^*aa^*bR_2$$

Paso 9: Sustituir R_4

$$R_4 = aR_1 + bR_2 + \lambda$$

Debido a que λ pertenece a R_4, toda la construcción garantiza que la cadena termina en `abaa`.
Resultado final simplificado:

$$R_0 = (a + b)^*abaa$$

viii. **Comparación de Métodos**

- **Complejidad:** Ambos métodos producen:

$$(a+b)^*abaa$$

  La expresión es clara y concisa.
- **Tiempo:** El método de eliminación es inmediato. El método de ecuaciones es largo y requiere múltiples sustituciones.
- **Facilidad:** El método de eliminación es más sencillo e intuitivo. El método de Arden es más formal pero más complejo.